Geavanceerde Computerarchitectuur
Session 1

Christoph Sanders & Wout Corijn

2025-02-10

1 Probleem

Dit labo dient als introductie tot GPU-programmering. We maken gebruik van een framework met CUDA om een programma te ontwikkelen dat een lijst parallel omkeert.

2 Methode

Voor te interageren met de NVIDEA GPU wordt het cuda framework gebruikt, dit biedt een abstractie voor het werken met deze GPU's en zal ervoor zorgen dat de output constant is ongeachte de hardware waar het opgedraaid wordt.

- Als eerste stap wordt op de GPU geheugen gealloceerd om de lijst in bij te houden.
- Daarna wordt de van de 'host' naar deze geheugenplek gekopieerd.
- Dan zal elke thread op basis van zijn eigen index een element in de lijst verwerken. Het zal dit item swappen met de overeenkomstige item op het einde van de lijst (er wordt een spiegelbeeld gevormd).
- Na deze stappen zal de aangepaste lijst terug gekopieerd worden naar de 'host' en het geheugen op de GPU wordt weer vrijgegeven.

3 Resultaat

De uitvoer bevestigt het verwachte gedrag: de array wordt succesvol omgekeerd. Alleen het verminderen van de threads zal de correctheid van resultaat beïnvloeden, als er minder threads zijn dan de helft van de lijst dan zal het midden van de lijst niet worden omgedraaid. Het vergroten van het aantal threads gaat de output niet beïnvloeden door de 'if' statement in de functie die de thread zal uitvoeren.

4 Conclusie

Taken zoals deze, die parallel kunnen uitgevoerd worden zijn geschikt voor het uitvoeren op de GPU. Dit is sneller dan het sequentieel uitvoeren op de CPU.

5 Extra Vraag

Maximaal aantal threads dat het systeem kan verwerken is 1024 threads per block, als hier over wordt gegaan dan zal de de lijst gewoon niet verwerkt worden en zal deze oorspronkelijke lijst naar de output gekopieerd worden.

6 Appendix

```cpp
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void reverseArray(int *d_arr, int size) {
    int idx = threadIdx.x;
    int swapIdx = size - 1 - idx;

    if (idx < size / 2) { // Zorg ervoor dat we niet over het midden heen gaan
        int temp = d_arr[idx];
        d_arr[idx] = d_arr[swapIdx];
        d_arr[swapIdx] = temp;
    }
}

int main() {
    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);
    printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);
    printf("Max Blocks per Grid: %d, %d, %d\n",
        prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);



    const int size = 200;
    int h_arr[size];
    for (int i = 0; i < size; i++) {
        h_arr[i] = i;
    }

    int *d_arr;

    // geheugen alloceren op GPU
    cudaMalloc((void**)&d_arr, size * sizeof(int));

    // kopieren van host naar device
    cudaMemcpy(d_arr, h_arr, size * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel met (size/2) threads (1 per element)
    reverseArray<<<1, size/2>>>(d_arr, size);

    // kopieren van resultaat terug naar host
    cudaMemcpy(h_arr, d_arr, size * sizeof(int), cudaMemcpyDeviceToHost);

    // Free GPU memory
    cudaFree(d_arr);

    printf("Reversed List: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", h_arr[i]);
    }
    printf("\n");

    return 0;
}
```